

Reinforcement Learning for Games

Q-Learning Tic-Tac-Toe Agent — Project Report

Intern Name	Manas Deo
Intern ID	CITS7621
Project Name	Reinforcement Learning for Games
Program Duration	4-Week Internship Program
Domain	Machine Learning (Reinforcement Learning)

Project Scope: Build a Reinforcement Learning agent that learns to play Tic-Tac-Toe from scratch through trial-and-error (Q-Learning), without any labeled dataset — the agent's 'data' is its own gameplay experience, generated dynamically through thousands of self-played training games.

1. Project Overview

This project implements a complete Reinforcement Learning pipeline for a classic game, Tic-Tac-Toe. Unlike supervised learning, where a model learns from a fixed, labeled dataset, this agent learns purely by playing the game repeatedly and receiving rewards: **+1** for winning, **0.5** for a draw, **-1** for losing, and **-10** for an illegal move.

The agent uses tabular **Q-Learning** with an epsilon-greedy policy to balance exploring new moves against exploiting what it has already learned works well.

Note on data: This project has no static CSV dataset like a typical supervised ML project. Instead, the data is generated dynamically — every training episode (one full game) produces an experience record (state, action, reward, next state) that the agent learns from immediately. All 50,000 training episodes are also logged to a CSV file for analysis.

How It Works

Component	Description
State	The 3x3 board as a tuple of 9 cells (1=agent, -1=opponent, 0=empty)
Action	Choosing an empty cell index (0-8) to place a mark
Reward	+1 win, 0.5 draw, -1 loss, -10 illegal move
Policy	Epsilon-greedy: explores randomly early, exploits learned knowledge later
Learning Rule	Q-Learning Bellman update
Training Opponent	Plays uniformly random valid moves

2. Folder Structure

```
reinforcement-learning-for-games/
|
|-- README.md                # Project documentation
|-- requirements.txt          # Python dependencies
|-- .gitignore
|
|-- src/
|   |-- environment.py        # Tic-Tac-Toe game environment (from scratch)
|   |-- q_learning_agent.py    # Q-Learning agent (Q-table, epsilon-greedy)
|   |-- train_agent.py         # Trains the agent over 50,000 episodes
|   |-- evaluate_agent.py      # Evaluates trained agent vs random opponent
|   |-- play_game.py          # Interactive CLI: play against the agent
|
|-- models/
|   |-- q_table.pkl           # Trained Q-table (~7,500 state-action entries)
|
|-- outputs/
|   |-- plots/
|   |   |-- training_rewards.png # Moving-average reward curve
|   |   |-- win_rate_over_time.png # Win/draw/loss rate over training
|   |-- reports/
|   |   |-- training_log.csv      # Per-episode reward/result/epsilon log
|   |   |-- evaluation_report.txt # Final evaluation summary
|
|-- docs/
|   |-- project_report.md       # Week-wise project report
```

3. Setup & Run Instructions

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Run the full pipeline (from inside the project folder)
cd src
python train_agent.py      # trains the agent for 50,000 episodes (~1-2 min)
python evaluate_agent.py   # evaluates the trained agent over 2,000 games
python play_game.py        # play against the trained agent yourself!
```

4. Training & Evaluation Results

Evaluation Summary (2,000 greedy games vs. random opponent)

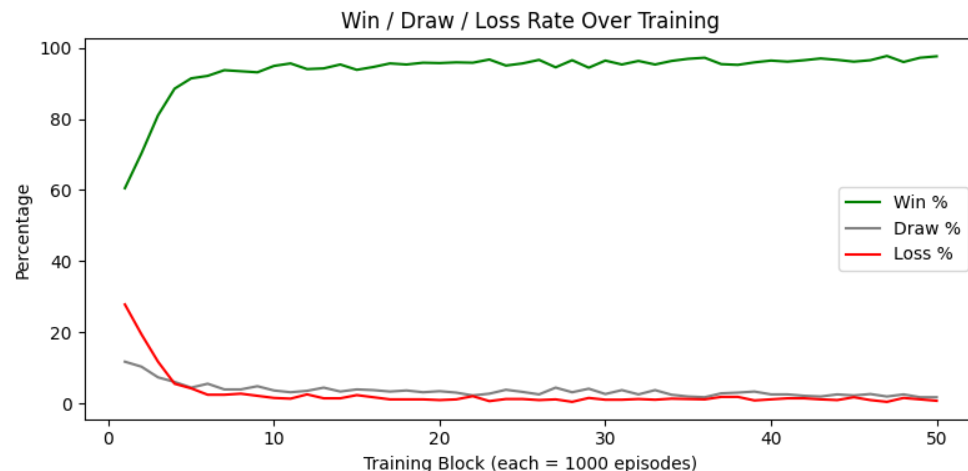
Metric	Result
Win Rate	98.8% (1976 / 2000 games)
Draw Rate	1.2% (24 / 2000 games)
Loss Rate	0.0% (0 / 2000 games)
Learned Q-table size	~7,529 state-action pairs

The agent learned to reliably win or draw against a random opponent, including correctly blocking opponent threats and taking winning moves when available — all without ever being explicitly told the rules of strategy, only the rules of the game itself (via rewards).

Training Reward Curve



Win / Draw / Loss Rate Over Training



5. Week-wise Project Breakdown

Week 1 - Problem Understanding & Environment Design

- Studied core Reinforcement Learning concepts: agents, environments, states, actions, rewards, policies, and the exploration/exploitation trade-off.
- Selected Tic-Tac-Toe as the game environment for fast tabular training.
- Built `environment.py` from scratch: board representation, valid-move detection, win/draw checking, and a step function following the standard RL interface.

Week 2 - Agent Design (Q-Learning)

- Built `q_learning_agent.py` implementing tabular Q-Learning with a Q-table, epsilon-greedy action selection, and the Bellman equation update rule.
- Added epsilon decay so the agent explores heavily early on and exploits its learned knowledge later in training.

Week 3 - Training & Evaluation

- Built `train_agent.py` to train the agent over 50,000 episodes against a random-move opponent.
- Logged every episode's reward, result, and epsilon value for analysis.
- Plotted the moving-average reward curve and win/draw/loss rate over training.
- Built `evaluate_agent.py` to test the fully-trained agent (greedy, no exploration) over 2,000 evaluation games.

Week 4 - Interactive Play & Documentation

- Built `play_game.py`, an interactive CLI game where a human can play Tic-Tac-Toe against the trained agent in real time.
- Organized the entire project into a clean, GitHub-ready folder structure with full README, requirements file, and this report.

Key Learnings

- Reinforcement Learning does not need a labeled dataset - the agent generates its own training signal (reward) purely through interaction with the environment.
- The epsilon-greedy strategy is essential: too little exploration and the agent gets stuck with a suboptimal policy; too much and it never converges in a reasonable time.
- Because Tic-Tac-Toe has a small, finite state space, tabular Q-Learning is sufficient - larger games would need function approximation (e.g. Deep Q-Networks).

Future Improvements

- Train the agent via self-play (against a copy of itself) for a stronger, more general strategy.
- Extend to more complex games (e.g. Connect Four) where a Deep Q-Network (DQN) would be needed instead of a plain Q-table.
- Add a simple web or GUI interface instead of the CLI for playing against the agent.

6. Full Source Code

src/environment.py

```
"""
environment.py
-----
A simple Tic-Tac-Toe game environment, built from scratch, that the
Reinforcement Learning agent interacts with.

Board representation:
  A 3x3 board is stored as a flat tuple of 9 cells (index 0-8):
    0 -> empty cell
    1 -> Agent's mark ("X")
   -1 -> Opponent's mark ("O")

  Board positions map like this:
    0 | 1 | 2
    ---
    3 | 4 | 5
    ---
    6 | 7 | 8

This environment follows the classic RL interface:
  state = env.reset()
  next_state, reward, done, info = env.step(action)

Rewards (from the AGENT's point of view, agent = player 1 / "X"):
  +1 -> agent wins
  -1 -> agent loses
  0.5 -> draw
  0 -> game continues (no reward yet)
 -10 -> illegal move (chosen a cell that is already filled)
"""

import random

AGENT = 1
OPPONENT = -1
EMPTY = 0

# All possible winning lines (rows, columns, diagonals) by cell index
WIN_LINES = [
    (0, 1, 2), (3, 4, 5), (6, 7, 8), # rows
    (0, 3, 6), (1, 4, 7), (2, 5, 8), # columns
    (0, 4, 8), (2, 4, 6), # diagonals
]

class TicTacToeEnv:
    """A minimal Tic-Tac-Toe environment for tabular Q-learning."""

    def __init__(self):
        self.board = [EMPTY] * 9
        self.done = False
        self.winner = None

    def reset(self):
        """Resets the board to empty and returns the initial state."""
        self.board = [EMPTY] * 9
        self.done = False
        self.winner = None
        return self.get_state()

    def get_state(self):
        """Returns the current board as an immutable tuple (used as a Q-table key)."""
        return tuple(self.board)

    def get_valid_actions(self):
        """Returns a list of empty cell indices where a move can legally be made."""
        return [i for i, cell in enumerate(self.board) if cell == EMPTY]

    def check_winner(self):
```

```

"""
Checks the board for a winner.
Returns AGENT, OPPONENT, 'draw', or None (game still ongoing).
"""
for a, b, c in WIN_LINES:
    line_sum = self.board[a] + self.board[b] + self.board[c]
    if line_sum == 3 * AGENT:
        return AGENT
    if line_sum == 3 * OPPONENT:
        return OPPONENT
if EMPTY not in self.board:
    return "draw"
return None

def step(self, action, player):
    """
    Applies a move for `player` (AGENT or OPPONENT) at cell `action`.
    Returns (next_state, reward, done, info) -- reward is always
    given from the AGENT's perspective, regardless of who just moved.
    """
    if self.board[action] != EMPTY:
        # Illegal move - heavily penalize and end the episode
        self.done = True
        return self.get_state(), -10, True, {"illegal_move": True}

    self.board[action] = player
    result = self.check_winner()

    if result == AGENT:
        self.done = True
        return self.get_state(), 1, True, {"winner": "agent"}
    elif result == OPPONENT:
        self.done = True
        return self.get_state(), -1, True, {"winner": "opponent"}
    elif result == "draw":
        self.done = True
        return self.get_state(), 0.5, True, {"winner": "draw"}
    else:
        # Game continues, no reward yet
        return self.get_state(), 0, False, {}

def render(self):
    """Prints the board in a human-readable 3x3 grid."""
    symbols = {AGENT: "X", OPPONENT: "O", EMPTY: " "}
    rows = []
    for r in range(3):
        cells = [symbols[self.board[r * 3 + c]] for c in range(3)]
        rows.append(" " + " | ".join(cells) + " ")
    print(("\\n" + "-" * 11 + "\\n").join(rows))

def random_opponent_move(env):
    """A simple opponent policy: picks a uniformly random valid move."""
    valid_actions = env.get_valid_actions()
    return random.choice(valid_actions)

```

src/q_learning_agent.py

```

"""
q_learning_agent.py
-----
A tabular Q-Learning agent for the Tic-Tac-Toe environment.

Q-Learning is a model-free Reinforcement Learning algorithm that learns
a Q-table mapping (state, action) pairs to expected future rewards.
The agent updates its estimates using the Bellman equation:


$$Q(s, a) \leftarrow Q(s, a) + \alpha * [ r + \gamma * \max_{a'}(Q(s', a')) - Q(s, a) ]$$


Where:
s      = current state (board configuration)
a      = action taken (cell index 0-8)
r      = reward received

```



```

s'      = next state after taking action a
alpha   = learning rate (how much new information overrides old)
gamma   = discount factor (how much future rewards matter)

```

The agent uses an epsilon-greedy policy during training: with probability epsilon it explores (random move), otherwise it exploits (picks the best known move so far). Epsilon decays over training so the agent explores a lot early on and exploits more as it learns.

```

import random
import pickle
from collections import defaultdict

```

```

class QLearningAgent:
    def __init__(self, alpha=0.3, gamma=0.9, epsilon=1.0,
                  epsilon_min=0.05, epsilon_decay=0.9995):
        self.alpha = alpha          # learning rate
        self.gamma = gamma          # discount factor
        self.epsilon = epsilon      # exploration rate (starts high)
        self.epsilon_min = epsilon_min # exploration never drops below this
        self.epsilon_decay = epsilon_decay # multiplicative decay per episode

        # Q-table: maps (state, action) -> estimated value. Defaults to 0.0
        # for any (state, action) pair we haven't seen yet.
        self.q_table = defaultdict(float)

    def choose_action(self, state, valid_actions, greedy=False):
        """
        Chooses an action using an epsilon-greedy policy.
        If greedy=True, always picks the best known action (used at
        evaluation/play time, not during training).
        """
        if not greedy and random.random() < self.epsilon:
            # Explore: pick a random valid action
            return random.choice(valid_actions)

        # Exploit: pick the action with the highest Q-value for this state
        q_values = [self.q_table[(state, a)] for a in valid_actions]
        max_q = max(q_values)
        # Break ties randomly among equally-good best actions
        best_actions = [a for a, q in zip(valid_actions, q_values) if q == max_q]
        return random.choice(best_actions)

    def update(self, state, action, reward, next_state, next_valid_actions, done):
        """Performs one Q-learning update using the Bellman equation."""
        current_q = self.q_table[(state, action)]

        if done or not next_valid_actions:
            target = reward # no future reward if the episode has ended
        else:
            future_qs = [self.q_table[(next_state, a)] for a in next_valid_actions]
            target = reward + self.gamma * max(future_qs)

        # Move the current estimate toward the target by a fraction (alpha)
        self.q_table[(state, action)] = current_q + self.alpha * (target - current_q)

    def decay_epsilon(self):
        """Reduces the exploration rate after each episode, down to a floor."""
        self.epsilon = max(self.epsilon_min, self.epsilon * self.epsilon_decay)

    def save(self, path):
        """Saves the Q-table (and hyperparameters) to disk using pickle."""
        with open(path, "wb") as f:
            pickle.dump({
                "q_table": dict(self.q_table),
                "alpha": self.alpha,
                "gamma": self.gamma,
                "epsilon": self.epsilon,
            }, f)

    def load(self, path):
        """Loads a previously saved Q-table from disk."""
        with open(path, "rb") as f:

```

```

        data = pickle.load(f)
        self.q_table = defaultdict(float, data["q_table"])
        self.alpha = data.get("alpha", self.alpha)
        self.gamma = data.get("gamma", self.gamma)
        self.epsilon = data.get("epsilon", self.epsilon)

```

src/train_agent.py

```

"""
train_agent.py
-----
Trains the Q-Learning agent by playing many episodes of Tic-Tac-Toe
against a random-move opponent. Over thousands of episodes, the agent
updates its Q-table and gradually learns which moves lead to wins.

Each full turn (agent move + opponent's reply) is treated as a single
Q-learning transition, since the environment always returns rewards
from the agent's perspective.

Outputs produced:
    models/q_table.pkl                -> trained Q-table
    outputs/reports/training_log.csv   -> per-episode reward/result/epsilon
    outputs/plots/training_rewards.png -> moving-average reward curve
    outputs/plots/win_rate_over_time.png -> win/draw/loss rate over training
"""

import os
import csv
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

from environment import TicTacToeEnv, random_opponent_move, AGENT, OPPONENT
from q_learning_agent import QLearningAgent

BASE_DIR = os.path.join(os.path.dirname(__file__), "..")
MODELS_DIR = os.path.join(BASE_DIR, "models")
PLOTS_DIR = os.path.join(BASE_DIR, "outputs", "plots")
REPORTS_DIR = os.path.join(BASE_DIR, "outputs", "reports")

N_EPISODES = 50000
LOG_EVERY = 500 # how often to print progress to the console

def play_one_episode(env, agent):
    """
    Plays a single training episode (agent vs random opponent) and
    updates the agent's Q-table along the way.
    Returns the total episode reward and the final result string
    ("agent", "opponent", "draw", or "illegal").
    """
    state = env.reset()
    done = False
    total_reward = 0
    result = None

    while not done:
        valid_actions = env.get_valid_actions()
        action = agent.choose_action(state, valid_actions)
        next_state, reward, done, info = env.step(action, AGENT)

        if info.get("illegal_move"):
            result = "illegal"

        if not done:
            # Opponent replies with a random valid move
            opp_action = random_opponent_move(env)
            next_state, reward, done, info = env.step(opp_action, OPPONENT)

        if info.get("winner") == "agent":
            result = "agent"
        elif info.get("winner") == "opponent":
            result = "opponent"

```

```

        elif info.get("winner") == "draw":
            result = "draw"

        next_valid_actions = env.get_valid_actions() if not done else []
        agent.update(state, action, reward, next_state, next_valid_actions, done)

        state = next_state
        total_reward += reward

    agent.decay_epsilon()
    return total_reward, result

def main():
    os.makedirs(MODELS_DIR, exist_ok=True)
    os.makedirs(PLOTS_DIR, exist_ok=True)
    os.makedirs(REPORTS_DIR, exist_ok=True)

    env = TicTacToeEnv()
    agent = QLearningAgent()

    log_rows = []
    wins = draws = losses = 0

    print(f"Training Q-learning agent for {N_EPISODES} episodes...\n")

    for episode in range(1, N_EPISODES + 1):
        reward, result = play_one_episode(env, agent)

        if result == "agent":
            wins += 1
        elif result == "draw":
            draws += 1
        elif result == "opponent":
            losses += 1

        log_rows.append({
            "episode": episode,
            "reward": reward,
            "result": result,
            "epsilon": round(agent.epsilon, 4),
        })

        if episode % LOG_EVERY == 0:
            recent = log_rows[-LOG_EVERY:]
            recent_wins = sum(1 for r in recent if r["result"] == "agent")
            win_pct = recent_wins / len(recent) * 100
            print(f"Episode {episode:6d} | epsilon={agent.epsilon:.3f} "
                  f"| win rate (last {LOG_EVERY}): {win_pct:5.1f}%")

    print(f"\nTraining complete. Overall: {wins} wins, {draws} draws, {losses} losses "
          f"out of {N_EPISODES} episodes.")

    # --- Save training log ---
    log_path = os.path.join(REPORTS_DIR, "training_log.csv")
    with open(log_path, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["episode", "reward", "result", "epsilon"])
        writer.writeheader()
        writer.writerows(log_rows)
    print(f"Saved training log to {log_path}")

    # --- Save trained Q-table ---
    model_path = os.path.join(MODELS_DIR, "q_table.pkl")
    agent.save(model_path)
    print(f"Saved trained Q-table to {model_path} "
          f"({len(agent.q_table)} state-action entries learned)")

    # --- Plot: moving-average reward over episodes ---
    rewards = [r["reward"] for r in log_rows]
    window = 500
    moving_avg = [
        sum(rewards[max(0, i - window):i + 1]) / len(rewards[max(0, i - window):i + 1])
        for i in range(len(rewards))
    ]
    plt.figure(figsize=(8, 4))

```

```

plt.plot(moving_avg, color="teal")
plt.xlabel("Episode")
plt.ylabel(f"Reward ({window}-episode moving average)")
plt.title("Training Reward Over Time")
plt.tight_layout()
plt.savefig(os.path.join(PLOTS_DIR, "training_rewards.png"))
plt.close()

# --- Plot: win/draw/loss rate per block of episodes ---
block_size = 1000
blocks = [log_rows[i:i + block_size] for i in range(0, len(log_rows), block_size)]
win_rates, draw_rates, loss_rates = [], [], []
for block in blocks:
    n = len(block)
    win_rates.append(sum(1 for r in block if r["result"] == "agent") / n * 100)
    draw_rates.append(sum(1 for r in block if r["result"] == "draw") / n * 100)
    loss_rates.append(sum(1 for r in block if r["result"] == "opponent") / n * 100)

plt.figure(figsize=(8, 4))
x = range(1, len(blocks) + 1)
plt.plot(x, win_rates, label="Win %", color="green")
plt.plot(x, draw_rates, label="Draw %", color="gray")
plt.plot(x, loss_rates, label="Loss %", color="red")
plt.xlabel(f"Training Block (each = {block_size} episodes)")
plt.ylabel("Percentage")
plt.title("Win / Draw / Loss Rate Over Training")
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(PLOTS_DIR, "win_rate_over_time.png"))
plt.close()

print("Saved training plots to outputs/plots/")

if __name__ == "__main__":
    main()

```

src/evaluate_agent.py

```

"""
evaluate_agent.py
-----
Loads the trained Q-table and evaluates the agent's performance by
playing many games (fully greedy, no exploration) against the random
opponent. Reports win/draw/loss rate and saves a summary report.

Run this AFTER train_agent.py has produced models/q_table.pkl
"""

import os

from environment import TicTacToeEnv, random_opponent_move, AGENT, OPPONENT
from q_learning_agent import QLearningAgent

BASE_DIR = os.path.join(os.path.dirname(__file__), "..")
MODELS_DIR = os.path.join(BASE_DIR, "models")
REPORTS_DIR = os.path.join(BASE_DIR, "outputs", "reports")

N_EVAL_GAMES = 2000

def play_eval_episode(env, agent):
    """Plays one fully-greedy episode (no exploration) and returns the result."""
    state = env.reset()
    done = False
    result = None

    while not done:
        valid_actions = env.get_valid_actions()
        action = agent.choose_action(state, valid_actions, greedy=True)
        state, reward, done, info = env.step(action, AGENT)

    if not done:

```

```

        opp_action = random_opponent_move(env)
        state, reward, done, info = env.step(opp_action, OPPONENT)

    if info.get("winner") == "agent":
        result = "agent"
    elif info.get("winner") == "opponent":
        result = "opponent"
    elif info.get("winner") == "draw":
        result = "draw"

    return result

def main():
    model_path = os.path.join(MODELS_DIR, "q_table.pkl")
    if not os.path.exists(model_path):
        raise FileNotFoundError(
            "No trained Q-table found. Run train_agent.py first to create "
            "models/q_table.pkl"
        )

    print(f"Loading trained agent from {model_path} ...")
    agent = QLearningAgent()
    agent.load(model_path)

    env = TicTacToeEnv()
    wins = draws = losses = 0

    print(f"Evaluating agent over {N_EVAL_GAMES} games (greedy policy, no exploration)...\n")

    for _ in range(N_EVAL_GAMES):
        result = play_eval_episode(env, agent)
        if result == "agent":
            wins += 1
        elif result == "draw":
            draws += 1
        elif result == "opponent":
            losses += 1

    win_pct = wins / N_EVAL_GAMES * 100
    draw_pct = draws / N_EVAL_GAMES * 100
    loss_pct = losses / N_EVAL_GAMES * 100

    report = (
        f"Evaluation over {N_EVAL_GAMES} games (greedy policy vs random opponent)\n"
        f"{'-' * 60}\n"
        f"Wins:    {wins:5d}    ({win_pct:5.1f}%) \n"
        f"Draws:   {draws:5d}    ({draw_pct:5.1f}%) \n"
        f"Losses:  {losses:5d}    ({loss_pct:5.1f}%) \n"
    )
    print(report)

    os.makedirs(REPORTS_DIR, exist_ok=True)
    report_path = os.path.join(REPORTS_DIR, "evaluation_report.txt")
    with open(report_path, "w") as f:
        f.write(report)
    print(f"Saved evaluation report to {report_path}")

if __name__ == "__main__":
    main()

```

src/play_game.py

```

"""
play_game.py
-----
Interactive command-line game: YOU (human) play Tic-Tac-Toe against
the trained Q-learning agent. The agent plays as "X" and moves greedily
(always picking its best known move); you play as "O".

Usage:
python src/play_game.py

```

```

"""

import os

from environment import TicTacToeEnv, AGENT, OPPONENT
from q_learning_agent import QLearningAgent

BASE_DIR = os.path.join(os.path.dirname(__file__), "..")
MODELS_DIR = os.path.join(BASE_DIR, "models")

def load_agent():
    model_path = os.path.join(MODELS_DIR, "q_table.pkl")
    if not os.path.exists(model_path):
        raise FileNotFoundError(
            "No trained Q-table found. Run train_agent.py first to create "
            "models/q_table.pkl"
        )
    agent = QLearningAgent()
    agent.load(model_path)
    return agent

def prompt_human_move(env):
    """Asks the human player to choose a valid empty cell (0-8)."""
    valid_actions = env.get_valid_actions()
    while True:
        try:
            move = int(input(f"Your move {valid_actions}: ").strip())
        except ValueError:
            print(" Please enter a number between 0 and 8.")
            continue
        if move in valid_actions:
            return move
        print(f" Invalid move. Choose one of: {valid_actions}")

def print_board_positions():
    """Shows the player which number corresponds to which board cell."""
    print("\nBoard cell positions:")
    print(" 0 | 1 | 2 ")
    print("-----")
    print(" 3 | 4 | 5 ")
    print("-----")
    print(" 6 | 7 | 8 \n")

def main():
    print("=" * 50)
    print(" TIC-TAC-TOE vs. Trained RL Agent")
    print(" You are 'O', the agent is 'X'. Agent moves first.")
    print("=" * 50)

    agent = load_agent()
    env = TicTacToeEnv()
    state = env.reset()
    print_board_positions()

    done = False
    while not done:
        # --- Agent's turn (goes first, greedy policy) ---
        valid_actions = env.get_valid_actions()
        agent_action = agent.choose_action(state, valid_actions, greedy=True)
        state, reward, done, info = env.step(agent_action, AGENT)

        print("\nAgent's move:")
        env.render()

        if done:
            break

        # --- Human's turn ---
        human_action = prompt_human_move(env)
        state, reward, done, info = env.step(human_action, OPPONENT)
        print("\nYour move:")

```

```
env.render()

# --- Announce result ---
print("\n" + "-" * 50)
if info.get("winner") == "agent":
    print("Result: The AGENT wins! Better luck next time.")
elif info.get("winner") == "opponent":
    print("Result: YOU WIN! You beat the trained agent.")
elif info.get("winner") == "draw":
    print("Result: It's a DRAW!")
print("-" * 50)

if __name__ == "__main__":
    main()
```